

Reading Document Report

Lab 05 – GUI Programming with JavaFX and Exception Handling

Course: Object-Oriented Programming

Lab: Lab 05 – GUI Programming with JavaFX and Exception Handling

Report: Reading Document

1. Introduction

This reading assignment focuses on JavaFX GUI development and Java exception handling. The objective is to understand how graphical user interfaces are built using JavaFX and how exceptions are used to make applications more robust and reliable.

The provided materials include JavaFX concepts, Scene Builder usage, custom exception creation, exception propagation, and best practices for handling runtime errors in software applications.

2. JavaFX GUI Programming

2.1 JavaFX Architecture

JavaFX applications are organized around three main components:

- **Stage:** the main application window.
- **Scene:** contains all graphical components.
- **Nodes:** UI elements arranged in a scene graph.

This architecture allows developers to separate interface design from application logic.

2.2 FXML and Scene Builder

FXML is an XML-based language used to define JavaFX interfaces.

Advantages:

- Easier UI design.
- Separation between interface and logic.
- Better maintainability.

Scene Builder provides a drag-and-drop environment that automatically generates FXML files, reducing development effort.

2.3 Painter Application

The Painter application demonstrates:

- Event handling.
- Mouse interaction.
- JavaFX containers.
- Controller implementation.

Main features:

- Draw using mouse dragging.
- Draw when mouse is pressed.
- Clear drawing area.
- Eraser functionality using white color.

3. AIMS Customer Application

The laboratory extends the AIMS project by creating a customer-oriented JavaFX application.

3.1 Store Screen

The Store Screen allows users to:

- View available media.
- Play DVDs and CDs.
- Add media to cart.

The interface consists of:

- Header section.
- View Cart button.
- Media display area using GridPane.

3.2 Cart Screen

The Cart Screen supports:

- Viewing selected media.
- Searching and filtering media.
- Removing media.
- Playing media.
- Viewing total cost.
- Placing orders.

JavaFX TableView and ObservableList are used to automatically synchronize data and user interface components.

4. Exception Handling

4.1 Importance of Exception Handling

Exception handling allows applications to:

- Detect errors.
- Prevent unexpected crashes.
- Provide meaningful feedback to users.
- Maintain stable execution.

Without proper exception handling, applications may terminate abruptly when runtime errors occur.

4.2 Checked and Unchecked Exceptions

Checked Exceptions

Checked exceptions are verified by the compiler. Examples:

- `IOException`
- `FileNotFoundException`
- `ClassNotFoundException`

Developers must either:

- Handle them using try-catch.
- Declare them using throws.

Unchecked Exceptions

Unchecked exceptions extend `RuntimeException`. Examples:

- `NullPointerException`
- `IllegalArgumentException`
- `NumberFormatException`
- `IndexOutOfBoundsException`

These exceptions are not checked during compilation.

5. PlayerException in AIMS

5.1 Purpose

The AIMS project introduces a custom exception called `PlayerException`. Its purpose is to handle media playback errors. Example situations:

- DVD length is invalid.
- CD length is invalid.
- Track length is invalid.

5.2 Exception Propagation

The exception may be generated in:

- `DigitalVideoDisc.play()`

- `Track.play()`
- `CompactDisc.play()`

`CompactDisc` catches exceptions from tracks and rethrows them to higher levels of the application. This technique is called exception propagation.

5.3 User Feedback

When an exception occurs:

- Error messages should be displayed.
- Exception details should be logged.
- The application should continue operating when possible.

Recommended methods include:

- `getMessage()`
- `toString()`
- `printStackTrace()`

6. Best Practices for Exception Handling

The reading materials recommend:

1. Validate input before processing.
2. Use custom exceptions for domain-specific problems.
3. Catch exceptions at appropriate layers.
4. Preserve exception information.
5. Avoid suppressing exceptions silently.
6. Provide meaningful error messages.
7. Use exception propagation when necessary.

Example:

```
if (getLength() <= 0) {  
  
    throw new PlayerException("Invalid media length");  
}
```

7. Personal Reflection

Through this reading assignment, I gained a better understanding of JavaFX application development and exception handling techniques.

The JavaFX framework simplifies graphical user interface development through Scene Builder and FXML, while exception handling improves software reliability by detecting and managing runtime errors.

The concept of custom exceptions such as `PlayerException` is particularly useful because it allows developers to create meaningful error handling mechanisms that are specific to application requirements.

8. Conclusion

This reading assignment provided valuable knowledge about JavaFX GUI programming and exception handling in Java.

The main lessons learned include:

- Designing interfaces using FXML and Scene Builder.
- Managing user interactions through controllers.
- Creating and handling custom exceptions.
- Using try-catch and throws effectively.
- Building reliable and maintainable software systems.

These concepts are essential for developing professional Java applications and were successfully applied in the AIMS project.